

A formal specification of TCAS with UML and OCL using USE

Course Code: CSC57501_SER37501_25FA
Course name: Formal Specification Methods
Date of submission: 12/01/2025

Kruti Shah
Department of Computer Science
Quinnipiac University
Hamden, USA
kpshah@quinnipiac.edu

Thenmozhi Sekar
Department of Computer Science
Quinnipiac University
Stu.id: 2672573
tseakr@quinnipiac.edu

Vignesh Balaji Ravichandran
Department of Computer Science
Quinnipiac University
Stud.id: 2672578
vravichandran@quinnipiac.edu

Abstract— The project Traffic Collision Avoidance System(TCAS) for aircraft is used to demonstrate the process and purpose of several phases in software development in critical systems. It was mainly used to describe the importance of formal specifications, requirements analysis, Unified Modeling Language (UML), Object Constraint Language (OCL), and the Unified Specification Environment (USE) tool. The aviation industry's safety-critical subsystem TCAS monitors air route intruders, so mathematical architecture like formal specifications should be used to verify and validate its perfect function under critical conditions. In this project, the TCAS was analyzed and translated into a structured UML class model, which includes essential system entities, their attributes, and their interactions. These structural class models were subsequently reinforced with OCL constraints, including invariants and pre- and post-conditions, to accurately represent real-life scenarios.

Once the system was constructed, its effectiveness was verified and validated using the USE tool. The software enables real-life modeling that includes invariant verification and the generation of object diagrams. USE automatically generates class diagrams after loading the model for design verification. Additionally, the OCL constraints are validated by generating object diagrams corresponding to the respective class diagrams, allowing for the identification and rectification of any invariant violations. The results include an initial class diagram design, a validated UML model within the USE environment, and a list of verified constraints. This work highlights the importance of mathematical architecture such as UML, OCL, and USE in the development of software systems, particularly for safety-critical applications like TCAS.

Keywords—TCAS, UML, OCL, Formal Specifications, Verification and Validation, USE tool

I. INTRODUCTION (FORMAL MODELING)

The TCAS is a critical safety system installed on modern aircraft to prevent midair collisions by monitoring surrounding air traffic and issuing timely avoidance advisories. Since TCAS actively interrogates nearby transponders in aircraft, computes bearing, distance, altitude, and closure rates, and triggers Traffic Advisories (TAs) or Resolution Advisories (RAs) within seconds of possible conflict, near perfection of the system is necessary to avoid massive catastrophes. The complexities of TCAS behavior—such as intruder detection, alert issuance, and the coordinated maneuvers of equipped aircraft—require rigorous verification and validation. Achieving perfection in the modeling stage is

essential to prevent failures in the system's design and execution, thereby ensuring flight safety. To address this need, formal modeling was introduced in the TCAS domain. A structured UML class model was developed to capture the essential system components, along with their responsibilities and relationships. The Object Constraint Language (OCL) was employed to formalize the constraints of the class model and to specify multiplicity conditions that cannot be visually represented in UML. The final UML/OCL model is then loaded into the USE tool to detect inconsistencies, identify constraint violations, and depict an object diagram of the class diagram. This approach guarantees the development of a safety-critical system by incorporating verification and validation at every stage of the creation process, leading to a system that is free from failures.

II. BACKGROUND

A. Traffic Collision Avoidance System(TCAS)

The TCAS is a safety system used in aircraft to reduce the risk of mid-air collisions by issuing timely avoidance advisories. TCAS continuously monitors the protected airspace area of a flight path for potential intruders installed with a transponder to determine their distance, relative altitude, bearing, and closure rate. Modern airliners use TCAS-II, which provides two types of alerts: Traffic Advisories (TAs), issued approximately 20–48 seconds before a potential conflict, and Resolution Advisories (RAs), issued 15–35 seconds before a predicted collision. TAs alert the cockpit of incoming traffic while RAs instruct the flight crew to climb, descend, or maintain altitude to avoid the intruder. When both aircraft are equipped with TCAS-II, the systems coordinate complementary maneuvers, such as one aircraft climbing while the other descends. Because TCAS influences critical pilot actions and overrides control in case of emergency situations, the system must be made to perfection and validated at each phase of creation.

B. Unified Modelling Language(UML)

The Unified Modeling Language (UML) is a standardized method for visually representing the structure and behavior of software systems. UML diagrams assist developers in understanding complex systems, identifying components, and effectively communicating design concepts prior to implementation. UML facilitates uniform documentation and minimizes ambiguity by visually representing classes, relationships, and interactions in a clear and structured

manner. This is particularly crucial in domains where safety is of utmost importance, such as aviation systems.

C. UML class models

UML class diagrams represent the structural backbone of a system by representing classes, their attributes, operations, and associations. Classes model real-world concepts, attributes capture essential properties, and operations represent system behaviors or functions. Associations define how classes interact with each other, while multiplicities specify number constraints on these relationships. In the context of TCAS, the UML class model consists of entities such as TCAS System, Aircraft, Intruder, Advisory, Pilot, and Alert and how they relate through monitoring and detection. Class models provide a structural support for both understanding and validation of a critical system.

D. Formal Methods

Formal methods use arithmetic to define, construct, and test software and hardware systems. They help to eliminate confusion, identify errors early on, and provide proofs of correctness, all of which are critical for high-assurance systems. Formal verification ensures that system rules, boundaries, and behaviors meet safety requirements and do not result in undesirable or dangerous outcomes in safety-critical scenarios such as air traffic control to avoid crashes. Formal approaches improve the accuracy of UML diagrams beyond what visual models can provide.

E. Object Constraint Language(OCL)

The Object Constraint Language (OCL) is a form of formal language used to define rules, invariants, preconditions, postconditions, and constraints on UML models. While UML diagrams describe system structure visually, OCL adds mathematical precision by including constraints that must always hold. Examples include multiplicity rules, safety requirements, attribute limitations, and cross-object relationships. For the TCAS model, OCL is used to implement critical properties such as the requirement that a TCAS system must have each advisory linked to an intruder to ensure no intruder is left with zero advisory or each advisory is triggered only after the minimum required time is met for the advisory. OCL ensures that the final model is both logically sound and unambiguous.

F. Unified Specification Environment(USE) Tool

The Unified Specification Environment (USE) is a tool designed for defining, analyzing, and validating UML and OCL specifications. It allows users to load UML class models, integrate OCL constraints, and generate class diagrams for the specified classes and their associations. The USE tool also detects constraint violations, supports object diagram creation, and helps identify inconsistencies that may not be visible in diagrams alone. In this project, USE is utilized to validate the TCAS UML/OCL model, detect structural or logical errors, and ensure that all constraints are satisfied. As a result, it supports rigorous verification and validation.

III. RELATED WORK

Role of UML Class Diagrams in Object-Oriented Software Development [1] examines how these diagrams are applied in

real-world projects and why Model-Driven Architecture (MDA) has not been widely adopted in the industry. Software Development examines how UML class diagrams are being used in real-world projects and why Model-Driven Architecture (MDA) has not reached industry adoption. The authors identified gaps between theory and practice, noting that class diagrams are used informally as sketches rather than as precise, detailed models ready for code generation. Tool limitations, such as incomplete or non-compliant generated code, are a major problem, along with the necessity for stronger integration between diagrams and executable systems. This paper informed the TCAS project about the importance of building a structurally sound and formally validated UML class model rather than relying on informal diagrams that are ambiguous or inconsistent.

Formal Methods: Practice and Experience [2] provides a broad survey of industrial applications of formal methods across safety-critical industries such as aerospace, finance, transportation, and security systems. Through their 62 case studies, the authors show that formal methods significantly improve system safety, correctness, and long-term reliability while reducing defects and maintenance expenses. However, the paper also identifies significant flaws such as tool complexity, a shortage of trained engineers, and misconceptions about cost. These insights directly support the TCAS project's need for OCL constraints and formal verification, as formal design is vital for safety-critical systems like collision avoidance. The case studies also validate the importance of early verification, which parallels the TCAS model's USE-tool-based analysis.

Using OCL for Verified Re-engineering, [3] explores how OCL can be utilized to support legacy system migration and verify transformation. The authors show that standard OCL lacks constructs for pointers, loops, exceptions, and some low-level language features, causing them to propose new extensions that make OCL sufficient enough for re-engineering of COBOL, C, Pascal, and other legacy languages. Their main contribution is putting OCL in the main picture as a formalism that can keep business rules consistent across different implementation platforms. TCAS modeling project use this concept of how OCL accurately captures necessary system logic, keeps constraints in place, and prevents logical inconsistencies from happening. All of these things are very important when modeling and verifying TCAS safety rules.

USE: A UML-based Specification Environment for Validating UML and OCL, [4] represents the USE tool and demonstrates how UML models and OCL constraints can be validated through automated snapshot generation, constraint checking, and interactive exploration. The authors use marriage rules as an example to show how USE finds violations, shows object diagrams, and checks invariants one step at a time. The tool has major flaws, like not being able to support many types of behavioral diagrams, but it is still one of the most widely used tools for validating conceptual models. The TCAS project makes use of the same USE tool to load, verify, debug, and validate the TCAS UML/OCL specification, ensuring its constraints operate properly.

LLM as a Code Generator in Agile Model-Driven Development [5] proposes integrating GPT-4 into an Agile development workflow to automatically generate executable Java and Python code from UML and OCL models. Their experiments show that GPT-4 produced a more functionally

correct, structurally consistent code with low complexity, validating the potential of AI-powered formal modeling. Although GPT-4 occasionally produced minor errors such as missing import statements, the study demonstrated the power of combining formal models with AI-driven automation. The result shows that systems like TCAS need precise UML and OCL specifications—without clear and rigorously validated models, automated reasoning and generation tools cannot function reliably.

Automated Requirements Formalization for Agile MDE [6] introduces an NLP- and ML-driven pipeline that transforms natural language requirements into UML and OCL models automatically. This method reduces error and minimizes the need for manual modeling by extracting classes, attributes, associations, and behavioral elements from plain English documentation. The approach achieved high accuracy, with F-scores reaching up to 0.97, demonstrating that automated formalization led to increased precision and early validation in safety-critical fields, such as aviation. The paper underscores the importance of formal specification in systems where errors could lead to catastrophic consequences, such as in TCAS intruder detection and advisory logic. TCAS underwent multiple stages of verification for the requirement documentation, along with corrections made at each stage to ensure a consistent and error-free system. This process is similar to the approach described in the paper, where AI receives verification of the generated UML through several stages to achieve maximum efficiency.

IV. METHODOLOGY

The methodology for the TCAS project consisted of three phases. Phase 1 focused on understanding the requirements and designing the UML class model. Phases 2 and 3 addressed OCL translation, as well as the verification and validation of the class model. Each phase contributed to the development of a thoroughly validated system, reflecting the steps of the software development lifecycle commonly used in the industry for creating critical systems.

1) Understanding Requirements

A thorough study of the TCAS system was conducted by reviewing the article "TCAS: A System for Preventing Midair Collisions" by W.H. Harman[7]. The aim was to understand the operational and safety requirements of the system. A list of core components, like the TCAS system and aircraft and intruder entities, was identified after analyzing the documents. Key behaviors such as continuous transponder interrogation, intruder detection, and the issuance of Traffic Advisories (TA) and Resolution Advisories (RA) were analyzed to determine what information the model must represent. A list of assumptions was created based on these requirements. From this analysis, essential attributes such as altitude, distance, bearing, closure rate, and relative vertical speed were extracted. This enabled the identification of correct system boundaries and clarified the structural relationship necessary between entities for modeling TCAS. By mapping each requirement to a corresponding UML concept (class, attribute, or association), a clear foundation was set for constructing the class model.

2) Designing the UML class model

After preparing a clear requirements documentation, a UML class diagram was designed to represent the static structure of the TCAS. The tool that was initially used to

design the class diagram was Draw.io[8], which is a website-based online platform with limited tools and features to design a class diagram. As the complexity of the class diagram increased through various stages, we transitioned to using StarUML[9]. This tool provided an editor that allowed us to design the diagram in a textual format, rather than relying on a drag-and-drop interface. The tool complexity was low, and the functionality was high.

The model includes the TCASSystem class responsible for monitoring aircraft, the Aircraft class representing each airplane's current state, and the Intruder class capturing potential conflict targets detected by the system. Attributes were assigned according to the domain specifications—for example, Aircraft includes altitude and verticalSpeed, while Intruder includes distance, bearing, relativeAltitude, and tauSec. Associations were modeled to reflect real-world interactions: the TCAS system monitors multiple aircraft, and each aircraft may detect several intruder objects. Multiplicities were added to enforce numerical constraints derived from the requirements. The model captures only structural elements but provides the necessary base for OCL constraint definition.

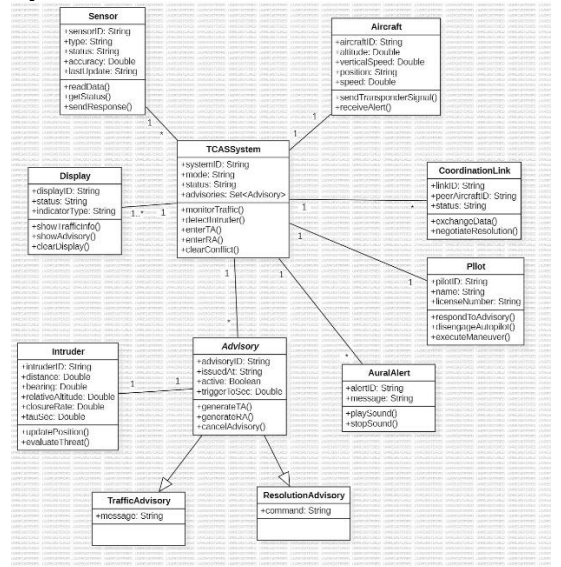


Figure 1. A class diagram designed using STARUML tool.

3) OCL Translation

The next step involved translating the UML diagram into the Object Constraint Language (OCL) to define the system's rules. This systematic process included several key components: First, each class from the UML diagram was represented in OCL format, detailing its attributes and data types. Next, OCL expressions illustrated associations, enabling precise references between objects, such as self.Aircraft or self.Intruders. Additionally, operations were specified with preconditions and postconditions to clarify their expected behavior. To further enhance clarity, natural language descriptions of the constraints were drafted initially before being converted into precise OCL invariants.

model TCAS_Assignment

```
-- CLASSES
class TCASSystem
attributes
  systemID : String
  mode : String
  status : String
operations
  TCASSystem(systemID : String, mode : String, status : String)
```

```

monitorTraffic() : String
detectIntruder() : String
enterTA(int : Intruder) : String
enterRA(int : Intruder) : String
clearConflict(int : Intruder) : String
end

class Aircraft
attributes
aircraftID : String
altitude : Real
verticalSpeed : Real
position : String
speed : Real
operations
Altitude(aircraftID : String, altitude : Real, verticalSpeed : Real,
position : String, speed : Real)
sendTempoAndSignal() : String
receiveAlert() : String
end

class Intruder
attributes
intruderID : String
distance : Real
bearing : Real
relativeAltitude : Real
closureRate : Real
tauSec : Real
operations
Intruder(intruderID : String, distance : Real, bearing : Real,
relativeAltitude : Real, closureRate : Real, tauSec : Real)
updatePosition() : String
evaluateThreat() : String
end

class Advisory
attributes
advisoryID : String
issuedAt : String
active : Boolean
triggerToSec : Real
operations
Advisory(advisoryID : String, issuedAt : String, active : Boolean,
triggerToSec : Real)
generateTA() : String
generateRA() : String
cancelAdvisory() : String
end

class TrafficAdvisory < Advisory
attributes
message : String
operations
TrafficAdvisory(message : String)
end

class ResolutionAdvisory < Advisory
attributes
command : String
operations
ResolutionAdvisory(command : String)
end

class Display
attributes
displayID : String
status : String
indicatorType : String
operations
Display(displayID : String, status : String, indicatorType : String)
showTrafficAndTA() : String
showAdvisory() : String
clearDisplay() : String
end

class AuralAlert
attributes
alertID : String
message : String
operations
AuralAlert(alertID : String, message : String)
playSound() : String
stopSound() : String
end

class CoordinationLink
attributes
linkID : String
peerAircraftID : String
status : String
operations
CoordinationLink(linkID : String, peerAircraftID : String, status : String)
exchangeData() : String
negotiateResolution() : String
end

class Pilot
attributes
pilotID : String
name : String
licenseNumber : String
operations
Pilot(pilotID : String, name : String, licenseNumber : String)
respondToAdvisory() : String
disengageAutoPilot() : String
executeManeuver() : String
end

class Sensor
attributes
sensorID : String
type : String
status : String
accuracy : Real
lastUpdate : String
operations
Sensor(sensorID : String, type : String, status : String,
accuracy : Real, lastUpdate : String)
readData() : String
getStatus() : String
sendResponse() : String
end

-- ASSOCIATIONS
association AircraftTCAS between
Aircraft[1] role aircraft
TCASSystem[1] role tcas
end

association SystemAdvisory between
TCASSystem[1] role system
Advisory[*] role advisories
end

association AdvisoryIntruder between
Advisory[1] role advisory
Intruder[1] role intruder
end

association SystemDisplay between
TCASSystem[1] role system
Display[1..*] role displays
end

association SystemAuralAlert between
TCASSystem[1] role system
AuralAlert[*] role alerts
end

association SystemCoordination between
TCASSystem[1] role system
CoordinationLink[*] role links
end

association PilotSystem between
Pilot[1] role pilot
TCASSystem[1] role system
end

association SystemSensor between
TCASSystem[1] role system
Sensor[1..*] role sensors
End
-- OCL constraints
constraints
-- Invariants

context Advisory
-- Every advisory must be associated with an intruder
inv: self.intruder <> null
-- Advisory trigger time must be non-negative
inv: self.triggerToSec >= 0.0

context TrafficAdvisory
-- Active TA must have trigger time in [20, 48] seconds
inv: not self.active or
(self.triggerToSec >= 20.0 and self.triggerToSec <= 48.0)

context ResolutionAdvisory
-- Active RA must have trigger time in [15, 35] seconds
inv: not self.active or
(self.triggerToSec >= 15.0 and self.triggerToSec <= 35.0)
-- RA command must be one of the allowed TCAS commands
inv: Set('CLIMB', 'DESCEND', 'MAINTAIN',
'INCREASE', 'CLIMB', 'INCREASE', 'DESCENT', 'LEVEL', 'OFF')
->includes(self.command)

context Intruder
-- Distance to intruder must be strictly positive
inv: self.distance > 0.0

context TCASSystem
-- At most one active advisory per intruder in this system
inv: self.advisories->select(a : a.active)
->forAll(a1, a2)
a1 <> a2 implies a1.intruder <> a2.intruder
)
-- System status must be one of the allowed values
inv: self.status = 'OK' or self.status = 'DEGRADED' or self.status = 'FAIL'

context Sensor
-- Sensor status must be one of the allowed values
inv: self.status = 'OK' or self.status = 'DEGRADED' or self.status = 'FAIL'

```

```

inv: self.status = 'OK' or self.status = 'DEGRADED' or self.status = 'FAIL'
-- Sensor accuracy must be within a valid range.
inv: self.accuracy > 0.0 and self.accuracy <= 1.0
-- Pre-Conditions

-- TCASSystem:enterTA(int : Intruder)
context TCASSystem:enterTA(int : Intruder) : String
-- Intruder must not be null
pre: int <> null
-- TA timing window [20, 48] seconds
pre: int.tauSec >= 20.0 and int.tauSec <= 48.0
-- No active advisory already exists for this intruder
pre: self.advisories
->select(a : a.active and a.intruder = int)
->isEmpty()

-- TCASSystem:enterRA(int : Intruder)
context TCASSystem:enterRA(int : Intruder) : String
-- Intruder must not be null
pre: int <> null
-- RA timing window [15, 35] seconds
pre: int.tauSec >= 15.0 and int.tauSec <= 35.0
-- No active advisory already exists for this intruder
pre: self.advisories
->select(a : a.active and a.intruder = int)
->isEmpty()

-- TCASSystem:clearConflict(int : Intruder)
context TCASSystem:clearConflict(int : Intruder) : String
-- Intruder must not be null
pre: int <> null

-- Display:showAdvisory()
context Display:showAdvisory() : String
-- There must be at least one active ResolutionAdvisory on this system
pre: self.system.advisories
->exists(a)
a.active and a.occlTypeOfResolutionAdvisory
)

-- Pilot:executeManeuver()
context Pilot:executeManeuver() : String
-- Pilot may execute maneuver only if an active RA exists on their system
pre: self.system.advisories
->exists(a)
a.active and a.occlTypeOfResolutionAdvisory
)

-- Post-Conditions (5)
-- TCASSystem:enterTA(int : Intruder)
context TCASSystem:enterTA(int : Intruder) : String
-- After enterTA, an active TrafficAdvisory exists for this intruder
post: self.advisories->exists(a)
a.active
and a.occlTypeOf(TrafficAdvisory)
and a.intruder = int
and a.triggerToSec = int.tauSec
)
-- Operation returns a confirmation message
post: result = 'TA_issued'

-- TCASSystem:enterRA(int : Intruder)
context TCASSystem:enterRA(int : Intruder) : String
-- After enterRA, an active ResolutionAdvisory exists for this intruder
post: self.advisories->exists(a)
a.active
and a.occlTypeOfResolutionAdvisory
and a.intruder = int
and a.triggerToSec = int.tauSec
and Set('CLIMB', 'DESCEND', 'MAINTAIN',
'INCREASE', 'CLIMB', 'INCREASE', 'DESCENT', 'LEVEL', 'OFF')
->includes(a.occlTypeOfResolutionAdvisory.command)
)
-- Operation returns a confirmation message
post: result = 'RA_issued'

-- TCASSystem:clearConflict(int : Intruder)
context TCASSystem:clearConflict(int : Intruder) : String
-- After clearConflict, there are no active advisories for that intruder
post: self.advisories
->select(a : a.active and a.intruder = int)
->isEmpty()
-- Operation returns a confirmation message
post: result = 'conflictCleared'

-- Display:showAdvisory()
context Display:showAdvisory() : String
-- After showAdvisory, the display is ON
post: self.status = 'ON'
-- Operation returns a confirmation message
post: result = 'advisoryShown'

-- AuralAlert:playSound()
context AuralAlert:playSound() : String
-- After playSound, the alert must have a non-empty message
post: self.message <> '' and self.message <> null
-- Operation returns a confirmation message
post: result = 'soundPlayed'

```

4) Verification and Validation with USE tool

The final phase involved the verification and validation of the OCL translation using the USE tool[10]. A single .use file is created using Notepad software, and the OCL translation is loaded into the .use file. The .use file is then uploaded into the environment for compilation and syntax checking. First, the classes and their associations are imported into the USE environment, and any errors that arise are corrected until the class model is successfully loaded.

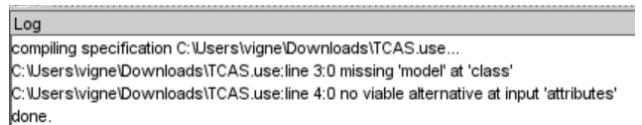


Figure 2 USE reports a parsing error because the .use file did not start with a required model block, causing the first class and its attributes to be rejected.

The error occurred because the .use file did not include the required “model ... end” block, causing USE to misinterpret the first class and reject the attributes section; the issue was resolved by placing all class and association definitions inside a “model TCAS ... end” structure and marking Advisory as an abstract class to correctly represent the intended inheritance hierarchy.

Log
compiling specification C:\Users\vigne\Downloads\TCAS.use...
C:\Users\vigne\Downloads\TCAS.use:line 3:0 missing 'model' at 'class'
C:\Users\vigne\Downloads\TCAS.use:line 4:0 no viable alternative at input 'attributes'
done.

Figure 3 USE displays errors because the file starts with a class instead of a model declaration, causing the first class and its attributes to be rejected.

The errors occurred because the USE file did not begin with the required model declaration, causing the parser to misread the first class and flag the attributes as invalid; the issue was fixed by adding a proper model ... end block at the top and enclosing all class definitions within it.

Log
compiling specification C:\Users\vigne\Downloads\TCAS.use...
done.
Model TCAS_Assignment (10 classes, 7 associations, 0 invariants, 32 operations, 0 pre-postconditions, 0 state machines)

Figure 4 Screenshot showing the UML class model successfully loaded in the USE environment with no errors, confirming that all classes, attributes, and associations were parsed correctly.

After successfully loading the model, an automatic snapshot of the class diagram is generated by the tool. This generated diagram is then used to cross-verify with the static class diagram created using the STARUML tool.

V. RESULTS AND ANALYSIS

The validated object configuration demonstrates that the core TCAS entities satisfy their respective invariants after refinement of the UML and OCL specifications. The object tcas1 : TCASSystem, defined with systemID “SYS001,” mode “TA_RA,” and status “OK,” satisfies the TCASSystem status invariant because “OK” is one of the allowed values in the status enumeration {OK, DEGRADED, FAIL}.

Class invariants	
Invariant	Satisfied
tcas1:TCASSystem	
systemID=null	
mode=null	
status=null	
Advisory::inv1	true
Advisory::inv2	true
Intruder::inv6	true
ResolutionAdvisory::inv4	true
ResolutionAdvisory::inv5	true
Sensor::inv10	true
Sensor::inv9	true
TCASSystem::inv7	true
TCASSystem::inv8	false
TrafficAdvisory::inv3	true
1 cnstr. failed. Inhere... Double click to open evaluation browser	

Figure 5 Invariant is false due to not being satisfied

The intr1 : Intruder object also satisfies all intruder constraints, with distance = 10.0 (strictly greater than zero), bearing = 45.0, relativeAltitude = -300.0, closureRate = 150.0, and tauSec = 25.0, confirming that the invariant enforcing positive distance is upheld. Together, these objects show that the primary monitoring and detection entities behave correctly under the defined model rules.

Class invariants	
Invariant	Satisfied
Advisory::inv1	true
Advisory::inv2	true
Intruder::inv6	false
ResolutionAdvisory::inv4	true
ResolutionAdvisory::inv5	true
Sensor::inv10	true
Sensor::inv9	true
TCASSystem::inv7	true
TCASSystem::inv8	true
TrafficAdvisory::inv3	true
1 cnstr. failed. Inhere... Double click to open evaluation browser	

Figure 6 Intruder invariant

The advisory objects also validate successfully under the corrected constraint set. The ta1 : TrafficAdvisory instance, with advisoryID “TA001,” issuedAt “2025-11-24T10:00,” active = true, triggerToSec = 25.0, and message “TRAFFIC,” satisfies the Traffic Advisory timing invariant because it is active and its triggerToSec value falls within the required window of 20 to 48 seconds. The object also satisfies structural constraints by maintaining a valid link to an intruder and ensuring triggerToSec ≥ 0 .

Class invariants	
Invariant	Satisfied
Advisory::inv1	false
Advisory::inv2	true
Intruder::inv6	true
ResolutionAdvisory::inv4	true
ResolutionAdvisory::inv5	true
Sensor::inv10	true
Sensor::inv9	true
TCASSystem::inv7	true
TCASSystem::inv8	true
TrafficAdvisory::inv3	false
2 cnstrs. failed. Inhere... Double click to open evaluation browser	

Figure 7 TA Invariant

Similarly, the ral1 : ResolutionAdvisory instance, with advisoryID “RA001,” issuedAt “2025-11-24T10:01,” active = false, triggerToSec = 30.0, and command “CLIMB,” satisfies all RA invariants. Because the advisory is inactive, the RA timing constraint holds trivially, and the command is valid within the allowed set {CLIMB, DESCEND, MAINTAIN, INCREASE_CLIMB, INCREASE_DESCENT, LEVEL_OFF}. These results confirm that both TA and RA advisory behaviors are modeled correctly and adhere to the safety requirements of TCAS.

Class invariants	
Invariant	Satisfied
Advisory::inv1	false
Advisory::inv2	false
Intruder::inv6	true
ResolutionAdvisory::inv4	false
ResolutionAdvisory::inv5	false
Sensor::inv10	true
Sensor::inv9	true
TCASSystem::inv7	true
TCASSystem::inv8	true
TrafficAdvisory::inv3	true
4 cnstrs. failed. Inhere...	
100%	

Figure 8 RA Invariant

After successfully completing all the constraints, a complete snapshot of the object diagram accurately reflects the class diagram of the UML model.

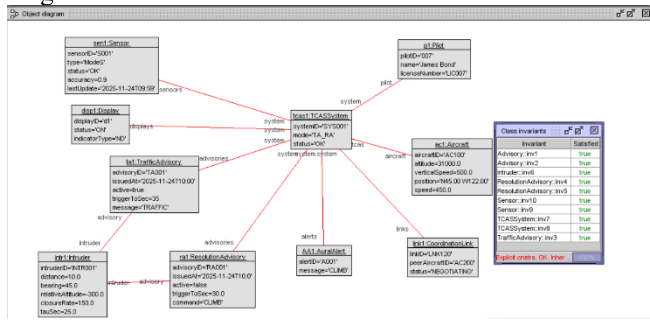


Figure 9 A complete object diagram with all invariants satisfied as true

VI. CONCLUSION

This project effectively developed a formally defined and thoroughly tested UML-based model of the Traffic Collision Avoidance System (TCAS). It assisted us in acquiring knowledge about a new aircraft model known as TCAS and provided practical experience in designing a UML class model. It demonstrated that the primary system architectures and safety regulations can be precisely represented through the integration of UML class modeling and OCL invariants. The resulting model fulfills all primary TCAS requirements, including aircraft monitoring, intruder detection, advisory generation, and timing behavior essential to safety. OCL was great at describing exact, clear constraints that couldn't be seen. This made it possible to formalize timing windows for alerts, criteria for intrusion validity, and limits on system

state. The USE tool played a crucial role in the verification process by identifying constraint violations, facilitating iterative refinement, and ensuring that the final model maintained logical consistency. Overall, the integration of UML, OCL, and USE provided a robust framework for analyzing and evaluating the TCAS domain.

VII. FUTURE WORK

Future improvements to this model could include making it more realistic and complex to accommodate more advanced analytical situations. A complete list of invariants and pre- and post-conditions is needed instead of just 20 constraints to fully model the aircraft's behavior. A tool is needed to translate the static class diagram into a dynamic model. An AI tool that can translate the requirements documentation into a class diagram. Additional system features, such as multi-threat handling and multi-resolution for several flights, can exhibit real-life scenarios. We can also generate a code from the diagram and verify its behavior in a code simulation.

REFERENCES

- [1] O. Nikiforova, J. Sejans, and A. Cernickins, "Role of UML class diagram in object-oriented software development," Scientific Journal of Riga Technical University: Computer Science. Applied Computer Systems, vol. 44, pp. 65–72, 2011.
- [2] J. Woodcock, P. G. Larsen, J. S. Fitzgerald, R. B. Banach, D. A. Bush, S. Hallerstede, P. G. Heath, M. C. Leuschel, B. Peleska, S. A. Sekerinski, and J. Bicarregui, "Formal methods: Practice and experience," ACM Computing Surveys, vol. 41, no. 4, pp. 1–41, 2009.
- [3] K. Lano and H. A. Siala, "Using OCL for verified re-engineering," in Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion '24), Linz, Austria, 2024, pp. 1–22.
- [4] M. Gogolla, F. Büttner, and M. Richters, "USE: A UML-based specification environment for validating UML and OCL," Science of Computer Programming, vol. 69, no. 1–3, pp. 27–34, 2007.
- [5] T. R. Dean, U. S. Reddy, A. E. Melton, and colleagues, "Applications of formal methods," Reading Assignment – CSC575/SER375, pp. 1–12, Course Material.
- [6] N. A. Qureshi, "Automated requirements formalisation for Agile MDE," Reading Assignment – CSC575/SER375, pp. 1–10, Course Material.
- [7] W. H. Harman, "TCAS: A system for preventing midair collisions," in IEEE Digital Avionics Systems Conference (DASC), pp. 1–10, 1988.
- [8] diagrams.net, "Draw.io – Online Diagramming Tool." Accessed: Nov. 30, 2025. [Online]. Available: <https://www.diagrams.net/>
- [9] MKLabs, "StarUML – UML Modeling Tool." Accessed: Nov. 30, 2025. [Online]. Available: <https://staruml.io/>
- [10] M. Gogolla et al., "The USE Tool – UML-Based Specification Environment." Accessed: Nov. 30, 2025. [Online]. Available: <https://useocl.github.io/>